# Take-Home Assignment: Platform Engineer

## The scenario

You've just joined a 5-engineer startup as their first platform engineer.

The team builds a web performance monitoring product. Customers embed a small JS SDK on their sites; it reports page performance events (LCP, page URL, session info) back to the company's infrastructure. Customers log in to a dashboard to see how their pages perform and to configure A/B tests.

Three services power this today:

- A Python (FastAPI) HTTP API that ingests events from the SDK and serves configuration back to it
- A Go background worker that consumes events from a queue and computes rolling aggregates
- A small static frontend, the customer-facing dashboard

They currently deploy by SSHing into a single VPS and running things under `screen`. There's no shared deploy workflow, no observability beyond `tail -f`, no rollback, no staging. When something breaks, whoever was last in the terminal figures it out.

Your job in week 1 to 2 on the job: build the minimum platform you'd hand them so they can stop SSHing into prod. As part of that, you'll also build minimal versions of the three services. See below.

## The task

Build the platform, plus minimal versions of the three services it runs.

You define what "minimum viable" means for the platform. That decision is part of what we're evaluating; over-engineered and under-built are both wrong answers. The apps are the substrate, not the deliverable. If you find yourself spending four hours on the dashboard's CSS, stop.

### Constraints

- The whole thing must run on a laptop. I'm not free to use any tool that you think could be helpful.
- The platform must support adding a fourth service in under 15 minutes, without platform changes. Document how.

### Stack

Open. Pick what you'd actually want to operate at a 5-person company.

### A note on AI

Use any AI tools you want. We assume you will. We're not testing whether you can type code. We're testing judgment, taste, and operability.

## The three services

You write all three services yourself. We are not providing any starter code, Dockerfiles, manifests, or scaffolding. Producing the apps is part of the assignment, and it's how we know your platform abstractions actually fit something you understand.

Keep them small. Reasonable shapes are sketched below; feel free to simplify further. The apps are the substrate, not the deliverable.

**1. The Python API (FastAPI).** Two endpoints:

- `POST /events`: receives page performance events from the SDK. JSON shape: `site_id`, `page_url`, `lcp_ms`, `timestamp`, `session_id`. Pushes onto a queue.
- `GET /config/{site_id}`: returns SDK config for a site (active experiments, sampling rate). In-memory map or SQLite is fine.

**2. The Go worker.** Consumes events from the queue. Computes rolling aggregates per `(site_id, page_url)`: event count, p75 LCP, last-seen timestamp. Persists to a store of your choice. Postgres, Redis, SQLite all work.

**3. The frontend.** A static dashboard that calls the API to show: top pages by event volume per site, a p75 LCP trend, active experiments. Plain HTML + JS is fine. Don't build full SPA unless you're already fast at it.

The queue between API and worker is an architectural choice. Pick one and explain it in the design doc.

## What to submit

A public git repository containing:

**1. The platform and services.** A single command (`make up` or equivalent) should bring the system up with all three services deployed and observable. We should be able (in theory) to deploy a fourth service ourselves by reading your docs.

**2. A 5-minute screen recording (Loom or similar).** Walk us through: deploying a service, what your observability surfaces, inducing a failure of your choice, and recovering from it. The demo is non-negotiable.

**3. A design doc (docs/design.md).** Include:

- A one-paragraph description of each service's shape: its inputs, outputs, and where its state lives
- Stack choices, with one alternative considered per major component and why you rejected it
- The one decision you're least confident about
- What you deliberately didn't build, and what would trigger building it later

**4. Two operational artifacts:**

- A one-page **user guide** for the engineering team: how they use the platform day-to-day
- A one-page **runbook** for the failure you demonstrated in the recording

# Submission

- Push everything to a public GitHub repo. We'll look at commit history, so feel free to commit naturally; we're not grading on git hygiene.
- Put all documents under `/docs` as markdown.
- Upload the Loom (unlisted is fine) and link it from the README.
- Reply to this email with the repo link when you're ready.

The scoping decisions you make under self-managed time are part of what we're looking at. If you spent 10 hours, tell us that and we'll factor it in; we'd rather know.